

IMPORTANTE, CAMBIOS EN EL API Y USO DE TOKENS:

Lo que ya tienes

- **API de academias:**
Ya implementa autenticación OAuth2 → recibe y valida un **access_token** emitido por *algún proveedor de tokens* (aunque ahora mismo ese “proveedor” lo tendrás embebido o simulado dentro de tu propio backend).
- **App Android:**
Se autentica y recibe un **access_token + refresh_token**.
- **Backend-chat-openai:**
Recibe el token del usuario y lo reutiliza directamente para llamar al API de academias.



Lo que necesitas añadir (no rehacer)

No necesitas reconstruir todo; solo necesitas introducir un IdP real que centralice la emisión y validación de los tokens.

Esto no rompe tu API de academias, solo cambia quién genera los tokens.

♦ 1 IdP: el nuevo servicio de identidad (Auth Server)

Este será el punto único de emisión y validación de tokens.

Puede ser un servidor externo (Auth0, Keycloak, etc.) o un servicio propio, si lo prefieres construir tú.

Tendrá tres funciones:

1. Emitir tokens OAuth2/OIDC (para Android, con refresh incluido).
2. Permitir el token exchange (OBO), para que el chat obtenga tokens delegados.

3. Publicar JWKS (claves públicas) para que el API de academias valide los tokens firmados.

♦ 2 Android App

- Ya no pide el token al API de academias ni a otro endpoint tuyo.
- Ahora se loguea contra el IdP y obtiene:
 - **access_token** (válido unos minutos)
 - **refresh_token** (para renovar)
- Usa ese **access_token** para hablar con el **chat-backend** o con el **API de academias**.

👉 El flujo de la app apenas cambia — solo cambia la URL de login.

♦ 3 Backend-chat-openai

- Sigue recibiendo el token del usuario igual que ahora.
- Cuando necesita acceder al API de academias:
 - Llama al IdP (**/token**) con **grant_type=token-exchange**.
 - Obtiene un token delegado.
 - Usa ese token delegado en la llamada al API de academias.

👉 Solo añades un servicio nuevo en el backend (TokenExchangeService), no reescribes todo.

♦ 4 API de academias

- Ya valida tokens OAuth2, así que no hay que tocar la lógica principal.
- Solo debes apuntar la validación (**issuer**, **jwks_uri**, **audience**) hacia el nuevo IdP.
- Los endpoints siguen igual, los roles siguen igual.

👉 No es un cambio de código funcional, es un cambio de configuración (de qué servidor emite los tokens válidos).



En resumen práctico

Componente	Qué cambia	Qué se mantiene
Android App	Se autentica contra el IdP, no contra el API directamente.	Sigue usando access y refresh tokens.
Chat-backend	Añade llamada de token exchange al IdP antes de acceder al API.	Mantiene su flujo general.
API de academias	Cambia el issuer y jwks_uri a apuntar al IdP.	Toda la lógica OAuth actual sigue igual.
Nuevo IdP	Servicio central que emite, refresca y valida tokens.	—



¿Es un cambio grande?

👉 A nivel de arquitectura: sí, introduces un nuevo componente (el IdP).

👉 A nivel de código: no es un cambio grande.

Tú no tienes que reescribir tus APIs ni tus controladores:

- Solo cambiarás la fuente de tokens.
- Añadirás un servicio en el backend-chat.
- Ajustarás la configuración de validación de tokens en el API.

Todo lo demás (usuarios, roles, permisos, flujos de negocio) sigue igual.

En resumen claro

- ✓ Sí, el IdP es un nuevo servicio común.
- ✓ Lo usarán: Android, chat-backend y API de academias.
- ✓ Tu API actual no se rompe: solo cambia de quién acepta los tokens.
- ✓ El cambio no es grande a nivel de código, pero sí es importante conceptualmente: pasas de un modelo distribuido a un modelo centralizado de autenticación.

Si quieres, puedo enseñarte ahora cómo quedaría tu arquitectura final con este IdP añadido (en un diagrama simple) y qué endpoints concretos necesitaría cada componente (**/token**, **/userinfo**, **/token/introspect**, **/token/exchange**, etc.). ¿Quieres que te lo muestre así visualmente?

Endpoints clave que necesitas

♦ IdP (nuevo servicio de autenticación)

Propósito	Endpoint típico	Quién lo usa
Login / emisión de tokens	<code>/protocol/openid-connect/token</code>	Android
Refrescar token	<code>/protocol/openid-connect/token (grant_type=refresh_token)</code>	Android

Token Exchange (delegación OBO) /protocol/openid-connect/token
(grant_type=token-exchange) Chat-backend

Validación de tokens (firma pública) /protocol/openid-connect/certs (JWKS) API de
Academias

Opcional: info del usuario /protocol/openid-connect/userinfo Android o Chat-backend

🧠 Qué hace cada componente ahora

📱 Android App

Llama directamente al IdP para loguear.

Recibe:

access_token (corto)

refresh_token (largo)

Usa el access_token en todas las llamadas al chat o al API.

Si el token caduca o faltan <3 min, refresca antes de enviar.

🤖 Backend-Chat-OpenAI

Recibe el mensaje del usuario con su access_token.

Si necesita datos del API:

Hace un Token Exchange en el IdP para obtener un `access_token_delegado`.

Usa ese token delegado para acceder al API de Academias.

Si el exchange falla (token de usuario caducado) → responde 401 para que Android refresque y reintente.

Puede cachear el token delegado 2–5 min.

 API de Academias

Ya valida tokens OAuth2, así que no cambia su lógica.

Solo debes configurar:

`issuer` = URL del IdP

`jwks_uri` = URL de las claves públicas

El token delegado le llega firmado, y puede ver:

`sub` (usuario)

`act` (actor = chat-backend)

aud (audience = academies-api)



Ventajas de este modelo

- ✓ Un solo punto de identidad (IdP) → más limpio, más seguro.
- ✓ No necesitas guardar refresh tokens en el chat ni en el API.
- ✓ Tokens cortos y controlados → menos exposición.
- ✓ Puedes escalar: en el futuro, otras apps (web, workers, etc.) usarán el mismo IdP.
- ✓ Mínimo cambio de código: solo configuración + un servicio pequeño en el chat-backend.

¿Quieres que te prepare ahora el docker-compose para montar el IdP (Keycloak) con la configuración inicial (realm academia, y los tres clientes: android-app, chat-backend, academies-api)?

Así podrías probar el flujo completo en local.

Tu backend-chat-openai recibe una petición **POST** del cliente (Android) con:

Authorization: Bearer <access_token_usuario>

-
- Hasta ahora, cuando el chat necesitaba llamar al API de academias, reutilizaba ese mismo token para autenticarse allí.

👉 Esto funciona, pero tiene los inconvenientes que comentamos: caducidad, refresh tokens, poca trazabilidad, etc.

Nueva versión (con Token Exchange)

Sí, el cambio correcto es este 👉

- 1 El backend-chat recibe el POST del usuario igual que siempre, con su **access_token** actual en el header:

Authorization: Bearer <access_token_usuario>

- 2 En lugar de usar ese token directamente para hablar con el API de academias, el backend-chat ahora hace una llamada al IdP (tu servidor de autenticación, por ejemplo Keycloak) para pedir un token delegado:

POST /protocol/openid-connect/token

Content-Type: application/x-www-form-urlencoded

grant_type=urn:ietf:params:oauth:grant-type:token-exchange

subject_token=<access_token_usuario>

audience=academies-api

client_id=chat-backend


```
client_secret=<secreto_chat_backend>
```

- 3 El IdP responde con un nuevo **access_token** delegado, válido para el **academies-api** (normalmente 2–5 min de vida).
- 4 El backend-chat usa ese token delegado para hacer la llamada al API de academias:

Authorization: Bearer <access_token_delegado>

- 5 El API de academias valida ese token como siempre, ve que el **sub** (subject) sigue siendo el usuario real, y opcionalmente un claim **act** (actor) que indica que actuó el servicio **chat-backend**.

✓ Por tanto, sí:

El backend-chat ya no debe usar el token del usuario directamente, sino que debe pedir al IdP un token delegado (vía Token Exchange) y usar ese token para acceder al API de academias.

Si eso te queda claro, el siguiente paso es definir dónde y cómo implementar el intercambio de token dentro del backend-chat (normalmente un pequeño **TokenExchangeService** con cache de 2–5 minutos). ¿Quieres que te muestre ese trozo después?

Lo que tienes ahora

- 1 API de Academias

- Ya usa OAuth2 (probablemente tipo “resource server”).
- Valida tokens de usuario (JWT).
- Gestiona roles: profesor, admin, superadmin, etc.

2 App Android

- Se autentica contra el IdP (o el backend de autenticación actual).
- Obtiene **access_token** y **refresh_token**.
- Llama al **chat-backend** enviando su **access_token**.

3 Backend-chat-openai

- Recibe peticiones desde Android con el token del usuario.
- Cuando necesita datos, usa ese mismo token para llamar al API de Academias.
- Si el token caduca → error 401, y Android tiene que refrescarlo y volver a enviar.

Problema con el modelo actual

- El backend-chat depende de tokens de usuario vivos, lo cual complica:
 - caducidades y refrescos,
 - trazabilidad de quién llamó a quién,
 - posibles errores si el token caduca durante una conversación larga.
- El chat se convierte en un “proxy ciego” que simplemente reenvía tokens, sin poder tener control fino sobre qué puede hacer cada llamada al API.

✓ Lo que te propongo (encaja con tu setup actual)

🎯 Meta

Que tu backend-chat actúe “en nombre del usuario”, pero con su propio token delegado, emitido automáticamente por el IdP.

De esa forma:

- Android sigue igual (usa su access/refresh token).
- El API de Academias sigue igual (valida JWT estándar).
- Solo cambia cómo el chat obtiene el token antes de llamar al API.

🔄 Cómo se vería ahora el flujo

Android (usuario logueado)

| Authorization: Bearer <access_token_usuario>



Chat-backend

| ① Verifica token recibido

| ② Si necesita llamar al API:

| → Pide al IdP un nuevo token “delegado”

| (grant_type=token-exchange)

| usando el token del usuario como “subject_token”



Academies API

| Authorization: Bearer <access_token_delegado>



Responde datos

El resultado:

Tu chat sigue respondiendo igual, pero ahora:

- Tiene un control completo de los tokens que usa hacia el API.
- No necesita guardar refresh tokens de los usuarios.
- El IdP sabe que “el backend-chat actuó por el usuario X”.



Cómo implementarlo paso a paso (resumen técnico)

1. En tu IdP (Auth Server o Keycloak):

- Habilita el grant type
`urn:ietf:params:oauth:grant-type:token-exchange`.
- Crea los clientes:
 - `android-app` → tipo *public*
 - `chat-backend` → tipo *confidential*
 - `academies-api` → *resource server*
- Autoriza que `chat-backend` pueda hacer token-exchange hacia `academies-api`.

2. En tu backend-chat:

Crea un servicio **TokenExchangeService** que haga:

POST /token

grant_type=urn:ietf:params:oauth:grant-type:token-exchange

subject_token=<access_token_usuario>

audience=academies-api

client_id=chat-backend

client_secret=*****

-
- Guarda el token delegado (válido 2–5 min) en cache.
- Si el API devuelve 401 → rehace el exchange una sola vez.

3. En tu API de academias:

- No necesitas tocar casi nada:
Solo asegúrate de validar correctamente el **audience** y **issuer** del token.
El **sub** seguirá siendo el usuario real, y **act** (actor) el chat-backend.

4. En Android:

- Sigue autenticando igual.
- Antes de mandar mensajes al chat, revisa si el **access_token** caduca pronto y refresca si es necesario (como ya hablamos).

Beneficios concretos para ti

Aspecto	Antes	Con Token Exchange
---------	-------	--------------------

Caducidad	El chat depende del token del usuario	El chat obtiene tokens nuevos a demanda
Refresh token	Necesario en cliente y backend	Solo Android lo guarda
Auditoría	No se sabe si la acción vino del chat o del usuario	El token dice “usuario X actuando a través de chat-backend”
Seguridad	Token de usuario expuesto a más servicios	Tokens cortos y limitados por scope
Coste de mantenimiento	Alto (control manual de caducidades)	Bajo (IdP gestiona todo)

… En resumen:

Sí — esto encaja exactamente con tu situación actual.

Tu API de academias ya usa OAuth2, tu backend-chat ya recibe tokens de usuario, y lo único que necesitas añadir es el paso intermedio de “token exchange” en el backend-chat, sin cambiar la lógica de negocio que ya tienes.

¿Quieres que te prepare un ejemplo real de implementación (en Spring Boot) del `TokenExchangeService`, mostrando cómo haría el `POST /token` y cómo cachear el nuevo token?

¡IMPLEMENTAR flujo On-Behalf-Of (OBO) / Token Exchange, qué papel tiene el IdP, y qué debes cambiar en Android y en el backend-chat.

Qué es el IdP

El IdP (Identity Provider) es el servidor que autentica usuarios y emite tokens (**access_token**, **refresh_token**, etc.).

Es quien valida credenciales y controla los permisos y caducidades de cada token.

Ejemplos comunes: Keycloak, Auth0, AWS Cognito, Okta, Azure AD.

En tu arquitectura, el IdP es el punto central que gestiona la identidad de los usuarios y los permisos que tienen sobre tu API de academias.

Qué es el flujo On-Behalf-Of (OBO) / Token Exchange

Es un mecanismo de delegación segura, donde un servicio (en tu caso el **backend-chat**) actúa en nombre del usuario, pero sin guardar su refresh token ni manejar su sesión directamente.

Cómo funciona en tu proyecto

1 El usuario se autentica en la app Android

- Android se comunica con el IdP (p. ej. Keycloak) y obtiene:
 - un **access_token** (válido unos minutos)
 - un **refresh_token** (para renovar el access token cuando caduque)
- Solo Android guarda el refresh token.

2 El usuario escribe en el chat

Android envía el mensaje al **backend-chat**, con el header:

Authorization: Bearer <access_token_usuario>

-

3 El backend-chat hace el token exchange

- En lugar de usar directamente ese token del usuario, el **backend-chat** le pide al IdP un nuevo token delegado, específico para el **academies-api**.

Ejemplo de llamada:

POST /token

Content-Type: application/x-www-form-urlencoded

grant_type=urn:ietf:params:oauth:grant-type:token-exchange

subject_token=<access_token_usuario>

audience=academies-api

client_id=chat-backend

client_secret=<secreto_del_chat_backend>

scope=academies.read academies.write

El IdP devuelve un nuevo **access_token** delegado con vida corta (2-5 min) que representa:

“El usuario X actuando a través del servicio chat-backend”.

4 El backend-chat llama al API de academias

Ahora usa ese token delegado en las peticiones:

Authorization: Bearer <access_token_delegado>

-

5 El API valida y responde

- El API de academias valida el token, ve el usuario (**sub**) y el actor (**act** = chat-backend), y responde con los datos.

Qué ventajas tiene este flujo

- No guardas refresh tokens en el backend.
El chat no gestiona caducidades largas ni sesiones de usuario.
- Trazabilidad completa.
El token incluye tanto al usuario como al servicio que actuó.
- Scopes mínimos.
Puedes limitar permisos por API y TTL.
- Seguridad y control.
Si se revoca al usuario, el IdP corta el flujo en el próximo exchange.

Qué necesitas cambiar

En el backend-chat

1. Configurar en el IdP que el cliente **chat-backend** puede hacer **token-exchange** hacia el **academies-api**.
(Habilitar el grant type **urn:ietf:params:oauth:grant-type:token-exchange**.)
2. Implementar un **TokenExchangeService**:
 - Hace la petición al **/token** del IdP con el **access_token** del usuario.
 - Guarda el token delegado en una cache corta (2–5 min) por (**userId**, **scopeSet**).

3. Reintento controlado:

- Si el token delegado caduca → rehace el exchange una vez.
- Si el token del usuario ya no vale → responde 401 al Android (para que él refresque y reintente).

4. Logs claros:

- Guarda **userId**, **act**, **audience**, **scope**, **jti** en cada request al API para auditoría.

En la app Android

1. Guardar tokens localmente (**access_token**, **refresh_token**, **expires_at**, **issued_at**), en un almacenamiento seguro (DataStore cifrado o EncryptedSharedPreferences).

2. Crear un **TokenManager**:

- Método **getValidAccessToken()** que:
 - Lee el token actual.
 - Si faltan ≤ 3 min para expirar → usa el **refresh_token** para renovarlo.
 - Devuelve siempre un token válido.

3. Usar ese método antes de cualquier llamada:

- Antes de enviar mensaje al chat o llamar al API de academias.
- Así nunca se manda un token próximo a caducar.

4. Fallback en errores:

- Si el backend-chat devuelve 401 → el Android refresca el token y repite la petición.

Flujo resumido

Android (access_token usuario)

|

▼

Chat Backend

| ① comprueba cache

| ② si no hay token delegado → pide al IdP un nuevo token (token exchange)

▼

Academies API (usa access_token_delegado)

|

▼

IdP (gestiona tokens, autenticación, permisos)

Resultado final

- Android mantiene los tokens vivos (usa refresh anticipado).
- Chat-backend nunca guarda refresh tokens ni depende de ellos.

- IdP controla todos los tokens y las relaciones “usuario-servicio-API”.
- API de academias recibe tokens válidos, con trazabilidad completa y caducidades cortas.
- Sistema más seguro, limpio y predecible.

¿Quieres que te muestre ahora el JSON real de petición y respuesta del `/token` de un token-exchange típico (por ejemplo con Keycloak o Auth0)?

porque me hablas de qaroni? el keycloak entonces lo desarrollaría como un proyecto independiente que da servicio tanto al app android como al chat-back-end_?

ChatGPT Plus

Buena observación 🙌

Te explico por qué mencioné *Qaroní* (y cómo encaja Keycloak en tu arquitectura real):



Por qué mencioné *Qaroní*

Lo mencioné solo como referencia contextual, porque ya tienes varios proyectos (Academy, Chat-backend, Workers, Media, etc.) que forman parte de tu ecosistema de desarrollo.

No cambia nada funcional: era para ilustrar cómo se integraría un único servidor de identidad (Keycloak) con tus servicios.

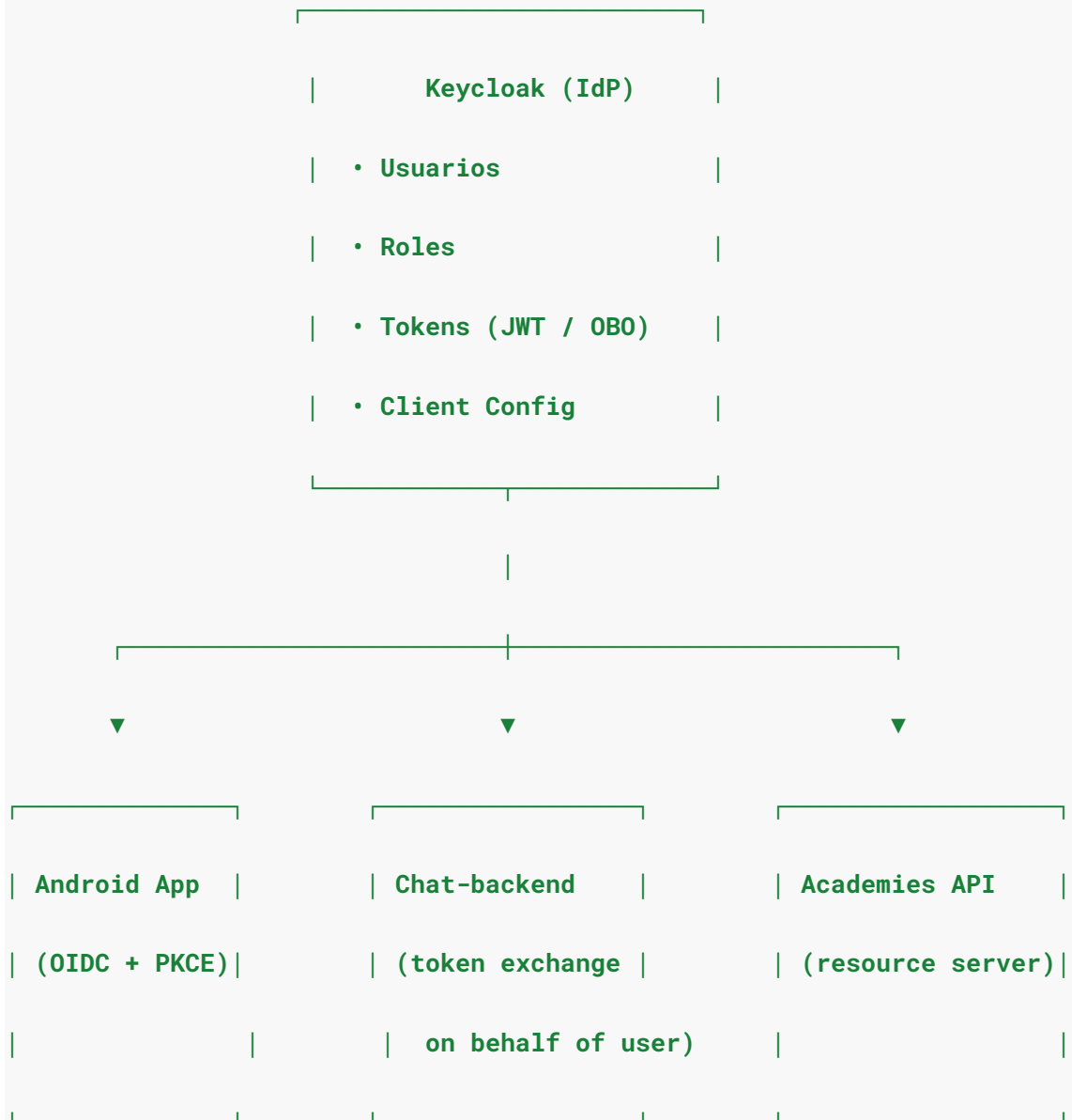
Así que olvídate del nombre “Qaroní” si no aplica directamente: puedes usarlo bajo el dominio que tú quieras.



Cómo se estructura realmente

Sí, lo ideal es que Keycloak sea un proyecto independiente (una pieza central común) que gestione toda la autenticación y emisión de tokens para *tus servicios y apps*.

En tu caso concreto, el esquema quedaría así:



⚙️ Qué papel tiene cada uno

🏗️ Keycloak

- Lo desarrollas (o despliegas) como un servicio independiente.

- Es tu Identity Provider (IdP).
- Define:
 - Usuarios y contraseñas
 - Roles / grupos (ej: profesor, admin, superadmin)
 - Clients (apps):
 - **android-app** → tipo *public client*
 - **chat-backend** → tipo *confidential client*
 - **academies-api** → *resource server*
- Emite tokens JWT firmados.
- Permite el Token Exchange (OBO):
chat-backend puede intercambiar el token del usuario por otro válido para **academies-api**.



Android App

- Se loguea contra Keycloak, obtiene sus tokens (**access_token**, **refresh_token**).
- Usa **access_token** para hablar con el **chat-backend** o con el **academies-api** directamente.



Chat-backend

- Recibe el **access_token** del usuario desde Android.

- Llama a Keycloak para hacer token exchange y obtener un token delegado.
- Con ese token delegado llama al **academies-api**.
- Devuelve la respuesta al usuario en el chat.

Academies API

- Actúa como resource server.
- Valida los tokens JWT que emite Keycloak (firma, issuer, audience).
- Aplica reglas por rol (**@PreAuthorize("hasRole('PROFESOR')")**, etc.).

Despliegue

Sí, lo ideal es tener Keycloak como un contenedor separado:
por ejemplo, en tu red local de desarrollo o en AWS (ECS, EC2 o incluso Docker Desktop).

Puedes tenerlo como proyecto independiente en tu workspace:

keycloak/

├ **docker-compose.yml**

├ **realm-export.json**

├ **.env**

└ **docs/**

Y tus otros proyectos (**chat-backend**, **academies-api**, **android-app**) lo usarán apuntando a:

<https://auth.tudominio.com/realms/academia>



Beneficio de hacerlo así

- Todos tus proyectos (Android, APIs, Workers futuros) comparten un único sistema de identidad central.
- Los roles, tokens, refresh y OBO están unificados y configurados una sola vez.
- Puedes crecer añadiendo más APIs o apps sin duplicar autenticación.
- Controlas toda la seguridad, los tokens y los logs desde un solo punto.

¿Quieres que te monte el ejemplo de estructura base de Keycloak como microservicio independiente (con `docker-compose.yml`, realm `academia`, y los tres clients preconfigurados: android, chat-backend y academies-api)?

Así tendrías una base lista para integrar.

Vamos al grano y te explico con claridad y sin rodeos qué implica esto y cuánto te cambia respecto a lo que ya tienes.



Lo que ya tienes

- API de academias:
Ya implementa autenticación OAuth2 → recibe y valida un `access_token` emitido por *algún proveedor de tokens* (aunque ahora mismo ese “proveedor” lo tendrás embebido o simulado dentro de tu propio backend).

- **App Android:**
Se autentica y recibe un **access_token** + **refresh_token**.
- **Backend-chat-openai:**
Recibe el token del usuario y lo reutiliza directamente para llamar al API de academias.

Lo que necesitas añadir (no rehacer)

No necesitas reconstruir todo; solo necesitas introducir un IdP real que centralice la emisión y validación de los tokens.

Esto no rompe tu API de academias, solo cambia quién genera los tokens.

♦ **IdP: el nuevo servicio de identidad (Auth Server)**

Este será el punto único de emisión y validación de tokens.

Puede ser un servidor externo (Auth0, Keycloak, etc.) o un servicio propio, si lo prefieres construir tú.

Tendrá tres funciones:

1. Emitir tokens OAuth2/OIDC (para Android, con refresh incluido).
2. Permitir el token exchange (OBO), para que el chat obtenga tokens delegados.
3. Publicar JWKS (claves públicas) para que el API de academias valide los tokens firmados.

♦ **Android App**

- Ya no pide el token al API de academias ni a otro endpoint tuyo.
- Ahora se loguea contra el IdP y obtiene:

- `access_token` (válido unos minutos)
- `refresh_token` (para renovar)
- Usa ese `access_token` para hablar con el `chat-backend` o con el `API de academias`.

👉 El flujo de la app apenas cambia — solo cambia la URL de login.

◆ ③ Backend-chat-openai

- Sigue recibiendo el token del usuario igual que ahora.
- Cuando necesita acceder al API de academias:
 - Llama al IdP (`/token`) con `grant_type=token-exchange`.
 - Obtiene un token delegado.
 - Usa ese token delegado en la llamada al API de academias.

👉 Solo añades un servicio nuevo en el backend (`TokenExchangeService`), no reescribes todo.

◆ ④ API de academias

- Ya valida tokens OAuth2, así que no hay que tocar la lógica principal.
- Solo debes apuntar la validación (`issuer`, `jwks_uri`, `audience`) hacia el nuevo IdP.
- Los endpoints siguen igual, los roles siguen igual.

👉 No es un cambio de código funcional, es un cambio de configuración (de qué servidor emite los tokens válidos).



En resumen práctico

Componente	Qué cambia	Qué se mantiene
Android App	Se autentica contra el IdP, no contra el API directamente.	Sigue usando access y refresh tokens.
Chat-backend	Añade llamada de token exchange al IdP antes de acceder al API.	Mantiene su flujo general.
API de academias	Cambia el issuer y jwtks_uri a apuntar al IdP.	Toda la lógica OAuth actual sigue igual.
Nuevo IdP	Servicio central que emite, refresca y valida tokens.	—



¿Es un cambio grande?

👉 A nivel de arquitectura: sí, introduces un nuevo componente (el IdP).

👉 A nivel de código: no es un cambio grande.

Tú no tienes que reescribir tus APIs ni tus controladores:

- Solo cambiarás la fuente de tokens.
- Añadirás un servicio en el backend-chat.
- Ajustarás la configuración de validación de tokens en el API.

Todo lo demás (usuarios, roles, permisos, flujos de negocio) sigue igual.



En resumen claro

- ✓ Sí, el IdP es un nuevo servicio común.
- ✓ Lo usarán: Android, chat-backend y API de academias.
- ✓ Tu API actual no se rompe: solo cambia de quién acepta los tokens.
- ✓ El cambio no es grande a nivel de código, pero sí es importante conceptualmente: pasas de un modelo distribuido a un modelo centralizado de autenticación.

✓ Tu idea es buena: “fasear” el cambio

👉 En lugar de meterte ya con Keycloak o un IdP completo, vas a mantener tu autenticación distribuida actual (el API de academias emite y valida tokens), pero vas a mejorar el chat-backend para que deje de reenviar el token tal cual y empiece a usar un token delegado interno.

Eso te da un paso intermedio perfecto:

- No rompes nada que ya funciona.
- No cambias el login ni las credenciales en Android.
- Preparas el terreno para que, cuando quieras migrar al IdP, solo cambie quién emite el token, no cómo lo usas.

⚙️ Cómo queda tu sistema en esta fase

♦ API de academias

- Sigue siendo el servidor de autenticación principal.
- Su método **POST /login**:
 - Comprueba **email** y **password** en tu MySQL de AWS.

- Devuelve **access_token** + **refresh_token**.
- Sigue validando tokens igual que ahora.

→ No tocas nada de su lógica.

♦ App Android

- Se loguea en el API de academias (como siempre).
- Guarda **access_token** y **refresh_token**.
- Usa el **access_token** en las llamadas:
 - al chat-backend
 - o directamente al API de academias.
- Si el token está próximo a caducar → refresca antes (como ya planificamos).

→ Tampoco cambia su flujo de login ni sus tokens.

♦ Chat-backend (aquí está la novedad)

1. Recibe el **access_token** del usuario.
2. En lugar de usar ese token directamente, genera un “token delegado interno”, por ejemplo, un JWT corto (firmado con tu clave privada del chat o una shared secret).
3. Ese token delegado:
 - Puede incluir los claims del usuario (**sub**, **rol**, etc.).
 - Tiene vida muy corta (2–5 minutos).

4. Cuando el chat necesita llamar al API de academias:

- Llama usando el token delegado, o si lo prefieres, sigue usando el token del usuario por ahora (hasta que montes el delegador completo).

👉 Lo importante es que el backend ya esté preparado para ese “paso intermedio”: que el chat no dependa estrictamente del token del usuario, sino que pueda emitir y manejar tokens propios.

Beneficio de hacerlo por fases

Fase	Qué consigues
Fase 1 (ahora)	Todo sigue funcionando. El login es igual. El API sigue emitiendo tokens. El chat empieza a manejar delegación básica.
Fase 2 (futura)	Introduces IdP (Keycloak/Auth0) como emisor de tokens. Solo cambias la fuente, no la lógica de consumo.
Fase 3	Desactivas el login interno del API y delegas todo el login y emisión de tokens al IdP.

👉 Es un camino evolutivo, controlado y sin bloqueos.

Recomendación técnica para esta primera fase

1. Define en el chat-backend un pequeño “TokenService”:

- Si quieres simular delegación, puedes generar un JWT corto (p. ej. 5 min) con una clave secreta distinta del API.

Ejemplo (Spring Boot, usando `io.jsonwebtoken` o `java-jwt`):

```
String delegatedToken = Jwts.builder()

    .setSubject(userEmail)

    .claim("role", userRole)

    .setExpiration(new Date(System.currentTimeMillis() + 5 * 60 * 1000))

    .signWith(SignatureAlgorithm.HS256, chatSecret)

    .compact();
```

-
2. Haz que todas las llamadas del chat al API de academias pasen por un método común, donde más adelante podrás sustituir tu token local por el token OBO real.
3. Documenta el cambio: en tu repo, deja claro que esta autenticación es temporal y que el chat se prepara para Token Exchange futuro.

En resumen

- ✓ Sí, es totalmente correcto el plan que propones.
- ✓ Es un paso lógico y seguro hacia una arquitectura con IdP central.
- ✓ No tienes que romper nada del API de academias.
- ✓ Solo preparas el chat para la futura delegación real.

¿Quieres que te muestre un ejemplo concreto de cómo implementar ese “Token Delegado temporal” en el chat-backend (en Spring Boot), para que ya lo dejes preparado para cuando pasemos a IdP?

Perfecto 🍌 — entonces vamos a dejarte una base sólida para esta “fase 1” (tu sistema actual, pero ya preparado para evolucionar al IdP después).

Te explico exactamente qué vas a hacer en el backend-chat-openai, con un ejemplo real y limpio en Spring Boot.

Objetivo

- Mantienes tu login actual en el API de academias.
- El chat-backend recibe el `access_token` del usuario (emitido por tu API).
- En lugar de reenviarlo tal cual, generas un token delegado interno (JWT corto).
- Usas ese token interno para hacer tus llamadas al API.
- El día que metas el IdP, este servicio solo cambia su fuente del token (no el resto del flujo).

Estructura mínima

`chat-backend-openai/`

```
|─ src/

|   |─ main/java/com/tuempresa/chat/security/

|   |   |─ TokenDelegationService.java

|   |   |─ JwtUtil.java

|   |   └─ ChatSecurityConfig.java

|   └─ ...

└─ application.yml
```



1. JwtUtil.java — generación de token delegado

```
package com.tuempresa.chat.security;

import io.jsonwebtoken.Jwts;

import io.jsonwebtoken.SignatureAlgorithm;

import io.jsonwebtoken.security.Keys;

import java.security.Key;

import java.util.Date;

import java.util.Map;

public class JwtUtil {

    private final Key key =
Keys.hmacShaKeyFor("clave-secreta-chat-32bytes-minimo-para-seguridad".getBytes());

    public String generateDelegatedToken(String subject, Map<String,
Object> claims) {

        long now = System.currentTimeMillis();

        return Jwts.builder()

            .setSubject(subject)

            .addClaims(claims)

            .setIssuedAt(new Date(now))
```

```
        .setExpiration(new Date(now + 5 * 60 * 1000)) // 5 min

        .signWith(key, SignatureAlgorithm.HS256)

        .compact();

    }

}
```

📌 Este token solo lo usará el chat-backend para comunicarse con tus APIs internas. Más adelante, este mismo servicio será reemplazado por el flujo real de *token exchange* con el IdP.

2. TokenDelegationService.java

```
package com.tuempresa.chat.security;

import org.springframework.stereotype.Service;
import java.util.Map;

@Service

public class TokenDelegationService {

    private final JwtUtil jwtUtil;

    public TokenDelegationService(JwtUtil jwtUtil) {

        this.jwtUtil = jwtUtil;
    }
}
```

```
}

    public String createDelegatedToken(String userEmail, String role) {

        Map<String, Object> claims = Map.of(

            "role", role,

            "actor", "chat-backend"

        );

        return jwtUtil.generateDelegatedToken(userEmail, claims);

    }

}
```



3. Uso en el flujo del Chat

Supón que tienes un método donde el chat llama al API de academias:

```
@RestController
```

```
@RequestMapping("/chat")
```

```
public class ChatController {
```

```
    private final TokenDelegationService tokenDelegationService;
```

```
    private final AcademiesApiClient academiesApiClient;
```

```
public ChatController(TokenDelegationService tokenDelegationService,
AcademiesApiClient academiesApiClient) {

    this.tokenDelegationService = tokenDelegationService;

    this.academiesApiClient = academiesApiClient;

}

@PostMapping("/ask")

public ResponseEntity<?>
processChatMessage(@RequestHeader("Authorization") String userToken,

                                                             @RequestBody ChatRequest
request) {

    // ❶ Extraer datos básicos del usuario (por ahora, puedes
decodificar el token original)

    String userEmail = TokenParser.extractEmail(userToken);

    String userRole = TokenParser.extractRole(userToken);

    // ❷ Crear token delegado (interno del chat)

    String delegatedToken =
tokenDelegationService.createDelegatedToken(userEmail, userRole);

    // ❸ Llamar al API de academias con este token

    var response = academiesApiClient.getCourses(delegatedToken);
```

```
// 4 Continuar con la lógica normal del chat...

return ResponseEntity.ok(response);

}

}
```

4. En el API de Academias (mínimos ajustes)

Por ahora, el API sigue validando los tokens como antes (los suyos propios).

El chat podría seguir mandando el token del usuario real mientras pruebas el flujo.

Más adelante, cuando introduzcas el IdP, solo cambiarás la verificación del **issuer** y los **audience**, pero el **Authorization: Bearer <token>** seguirá igual.

Conclusión

Etapa	Qué logras
Ahora	Mantienes tu autenticación distribuida, pero el chat ya genera tokens delegados internos.
Próximamente	Sustituyes tu generador local (JwtUtil) por un token exchange real con el IdP (Keycloak/Auth0).
Beneficio	No tocas el login, ni los roles, ni el API. Solo refactorizas el backend-chat cuando llegue la migración.

¿Quieres que te deje también el pequeño helper **TokenParser** (para extraer email y rol del token original) y un ejemplo de **AcademiesApiClient** para ver cómo inyectar el token delegado en las peticiones HTTP?

entonces hago esto: El chat-backend recibe el access_token del usuario (emitido por tu API). En lugar de reenviarlo tal cual, generas un token delegado interno (JWT corto). Usas ese token interno para hacer tus llamadas al API. ----- pero al hacer esto, en el api tendré que hacer cambios para que siga validando el token cuando se llame desde la capa de presentacion de android y para validar ahora tambien este nuevo token delegado interno?

⁷⁰⁰ Muy buena pregunta, Angel — y sí, justo ahí está el punto fino que hay que definir bien. Te explico exactamente qué pasa y qué opciones tienes para que no rompas nada y el cambio sea limpio.

Situación actual

Tu API de academias ahora valida un solo tipo de token:

👉 los que ella misma emite cuando el usuario se loguea (access_token + refresh_token).

Cuando la app Android llama directamente al API → todo bien.

Pero si ahora el chat-backend genera su propio token delegado, el API no sabrá validar ese token (porque está firmado con otra clave).

Entonces sí: necesitas hacer un pequeño cambio en el API de academias,

pero no grande, solo para permitir dos fuentes de validación:

- Tokens emitidos por el propio API (caso Android).
- Tokens emitidos por el chat-backend (caso delegación).

No tocas el flujo de login ni el resto del código, solo extiendes la validación JWT para aceptar tokens de otra firma / emisor.



Opción 1 – Validar ambos emisores (recomendada)

Tu API de academias, al validar un token, puede comprobar:

1. Quién es el issuer (iss) del JWT.
2. Qué clave (firma) se usó para firmarlo.

Entonces:

- Si `iss == "academies-api"` → validar con la clave `secret_api` (tu firma actual).
- Si `iss == "chat-backend"` → validar con la clave `secret_chat`.

En pseudo-código:

```
if (token.issuer == "academies-api") {  
    validateJwt(token, secretApi);  
} else if (token.issuer == "chat-backend") {  
    validateJwt(token, secretChat);  
} else {  
    throw new UnauthorizedException("issuer desconocido");  
}
```

✓ Así puedes seguir validando tokens de Android (tuyos) y los nuevos tokens del chat sin romper nada.



Opción 2 – Usar un claim especial en el token delegado

En tu `TokenDelegationService`, podrías incluir algo como:

```
.claim("delegated_by", "chat-backend")  
  
.claim("delegated_for", "academies-api")
```

Entonces, en tu API:

- Detectas ese claim y aplicas validación específica.
- Así distingues fácilmente entre “token normal de usuario” y “token delegado interno”.

Ejemplo:

```
if (token.hasClaim("delegated_by")) {  
  
    validateJwt(token, secretChat);  
  
} else {  
  
    validateJwt(token, secretApi);  
  
}
```



Opción 3 – Mantener validación única (temporal)

Si quieres hacerlo más simple todavía en esta primera fase:

- No generes todavía un token realmente distinto,
sino un “token delegado” firmado con la misma clave que usa tu API (`secret_api`).

- De esa forma, el API lo valida sin cambios.

⚠ No es ideal (no separas responsabilidades), pero sirve para probar el flujo sin tocar la validación actual.

Recomendación práctica

Para esta fase intermedia:

1. Mantén el mismo algoritmo y clave de firma (`secret_api`) tanto en el API como en el chat-backend.
2. Añade en el token delegado un claim como `"actor": "chat-backend"` para diferenciarlo.
3. Así el API lo validará igual que hasta ahora, pero podrás saber quién actuó.

📌 Ejemplo de token delegado:

```
{  
  
  "sub": "angel@example.com",  
  
  "role": "profesor",  
  
  "actor": "chat-backend",  
  
  "exp": 1739378400  
}
```

El API lo validará sin cambios, y si quieres, puedes añadir un simple `if (actor != null)` para saber que vino del chat.



En resumen

En esta fase	Qué hacer
✓ Android → API	sigue igual (token emitido por el API, validación normal)
✓ Chat → API	genera token delegado con claim actor=chat-backend
⚙️ API de academias	opcional: valida ambos emisores o acepta temporalmente la misma firma
🔒 Futuro	cuando montes el IdP, el API validará solo los tokens firmados por el IdP

Así mantienes compatibilidad completa sin tocar tu login ni tu app Android, y dejas preparado el API para que más adelante simplemente cambie el issuer (cuando migres al IdP).

Quieres que te pase el código exacto para esa validación doble (dos emisores) en Spring Boot, para que el API de academias acepte tokens tuyos y del chat?

Tú dijiste:

si pasame es código exacto para esa validación doble (dos emisores) pero en python ya que el api lo tengo desarrollado con python. Así mi API de academias aceptara tokens tuyos y del chat

ChatGPT Plus

Perfecto 🙌 — entonces vamos con la versión exacta en Python, pensada para tu caso real: tu API de academias ya emite y valida tokens JWT, y ahora quieres que también acepte los tokens delegados generados por el chat-backend.



Objetivo

Tu API debe poder aceptar dos tipos de tokens JWT:

1. Tokens emitidos por ella misma (cuando el usuario se loguea).
2. Tokens delegados emitidos por el chat-backend (con otro **issuer** o **actor**).



Supuestos base

- Usas JWT con firma HS256 (clave secreta).
- Tu API usa FastAPI, Flask o similar.
- Los tokens se envían por el header **Authorization: Bearer <token>**.

Ejemplo práctico con PyJWT (funciona igual en Flask o FastAPI)

Instala si no lo tienes:

```
pip install pyjwt
```



auth_utils.py

```
import jwt
```

```
from jwt import PyJWTError
```

```
from datetime import datetime, timezone
```

```
# Claves de firma (pueden venir de variables de entorno)

SECRET_API = "clave-supersecreta-api"

SECRET_CHAT = "clave-secreta-chat-backend"


ISSUER_API = "academies-api"

ISSUER_CHAT = "chat-backend"


def validate_jwt(token: str):

    """

    Valida un JWT que puede provenir del API (issuer=academies-api)

    o del chat-backend (issuer=chat-backend).

    Devuelve el payload si es válido o lanza excepción si no lo es.

    """

    try:

        # Decodificamos sin verificar aún, para ver el issuer

        unverified = jwt.decode(token, options={"verify_signature": False})

        issuer = unverified.get("iss")

    except PyJWTError:

        raise ValueError("Token inválido o mal formado")


    # Elegimos la clave según el emisor

    if issuer == ISSUER_API:
```

```
        secret = SECRET_API

elif issuer == ISSUER_CHAT:

    secret = SECRET_CHAT

else:

    raise ValueError(f"Issuer no reconocido: {issuer}")

try:

    # Validamos la firma, expiración, etc.

    payload = jwt.decode(

        token,

        key=secret,

        algorithms=["HS256"],

        issuer=issuer,

        options={"require": ["exp", "sub", "iss"]}

    )

    # Verificamos expiración manualmente (opcional)

    exp = datetime.fromtimestamp(payload["exp"], tz=timezone.utc)

    if exp < datetime.now(timezone.utc):

        raise ValueError("Token expirado")

    return payload
```

```
except PyJWTError as e:

    raise ValueError(f"Error al validar token: {e}")
```

Ejemplo de uso en un endpoint (FastAPI)

```
from fastapi import FastAPI, Depends, HTTPException, Header

from auth_utils import validate_jwt


app = FastAPI()


def get_current_user(authorization: str = Header(...)):

    if not authorization.startswith("Bearer "):

        raise HTTPException(status_code=401, detail="Token no válido")

    token = authorization.split(" ")[1]

    try:

        payload = validate_jwt(token)

        return payload

    except ValueError as e:

        raise HTTPException(status_code=401, detail=str(e))


@app.get("/courses")
```

```
def get_courses(user=Depends(get_current_user)):  
  
    # user contendrá el payload del JWT validado  
  
    return {  
  
        "user": user["sub"],  
  
        "role": user.get("role"),  
  
        "actor": user.get("actor", "frontend"),  
  
        "data": ["Curso 1", "Curso 2"]  
  
    }
```

Ejemplo de tokens válidos

Token emitido por el API (actual)

```
{  
  
    "sub": "angel@example.com",  
  
    "role": "profesor",  
  
    "iss": "academies-api",  
  
    "exp": 1739330000  
  
}
```

Token delegado emitido por el chat-backend

```
{  
  
    "sub": "angel@example.com",
```

```
"role": "profesor",

"actor": "chat-backend",

"iss": "chat-backend",

"exp": 1739330500

}
```

Ambos serán aceptados por la función `validate_jwt()` 👉

Solo cambiará la clave de verificación y el `issuer`.



En resumen

Token viene de	Firma con	Validado por
API de academias	<code>SECRET_API</code>	API (issuer = academies-api)
Chat-backend	<code>SECRET_CHAT</code>	API (issuer = chat-backend)



Así:

- Las peticiones desde Android seguirán funcionando exactamente igual.
- Las peticiones que provengan del chat también serán válidas, siempre que se firmen con `SECRET_CHAT`.
- En el futuro, cuando uses el IdP, solo tendrás que cambiar esta lógica para validar con la clave pública del IdP (JWKS) en lugar de las dos secretas locales.

¿Quieres que te muestre ahora cómo debe generar el chat-backend ese token delegado compatible con esta validación (en Python también, para que pruebes que encaja 100 %)?

Tú dijiste:

si pero el chat-backend lo tengo desarrollado con java springboot

ChatGPT Plus

Perfecto 🍌 — entonces dejamos tu API de academias en Python como está (ya preparada para validar ambos tokens),

y te paso ahora el código Java (Spring Boot) exacto que debe implementar el chat-backend para generar ese token delegado compatible con la validación que ya hicimos en Python.

Objetivo

El chat-backend:

- Recibe el **access_token** del usuario (emitido por el API de academias).
- Genera un token delegado corto (JWT), firmado con su propia clave (**SECRET_CHAT**).
- Ese token lleva **iss = "chat-backend"**, **sub** = email o userId, y opcionalmente **actor**.
- El API de academias (Python) lo validará correctamente con la clave **SECRET_CHAT**.

Dependencias

Asegúrate de tener en tu **pom.xml** la librería JWT:

```
<dependency>
```

```
    <groupId>io.jsonwebtoken</groupId>
```

```
    <artifactId>jjwt-api</artifactId>
```

```
    <version>0.12.5</version>
```

```
</dependency>
```

```
<dependency>

    <groupId>io.jsonwebtoken</groupId>

    <artifactId>jjwt-impl</artifactId>

    <version>0.12.5</version>

    <scope>runtime</scope>

</dependency>

<dependency>

    <groupId>io.jsonwebtoken</groupId>

    <artifactId>jjwt-jackson</artifactId>

    <version>0.12.5</version>

    <scope>runtime</scope>

</dependency>
```



Clase JwtDelegationService.java

```
package com.tuempresa.chat.security;

import io.jsonwebtoken.Jwts;

import io.jsonwebtoken.SignatureAlgorithm;

import io.jsonwebtoken.security.Keys;

import org.springframework.stereotype.Service;
```

```
import java.security.Key;

import java.util.Date;

import java.util.Map;

@Service

public class JwtDelegationService {

    // 👉 Usa la misma clave que definiste en el API de academias
    (SECRET_CHAT)

    private static final String SECRET_CHAT = "clave-secreta-chat-backend";

    private static final Key KEY =
Keys.hmacShaKeyFor(SECRET_CHAT.getBytes());

    // Configuración básica

    private static final String ISSUER = "chat-backend";

    private static final long EXPIRATION_MS = 5 * 60 * 1000; // 5 minutos

    public String generateDelegatedToken(String userEmail, String role) {

        long now = System.currentTimeMillis();

        Map<String, Object> claims = Map.of(

            "role", role,

            "actor", "chat-backend"
```

```
);

return Jwts.builder()

    .setIssuer(ISSUER)

    .setSubject(userEmail)

    .setClaims(claims)

    .setIssuedAt(new Date(now))

    .setExpiration(new Date(now + EXPIRATION_MS))

    .signWith(KEY, SignatureAlgorithm.HS256)

    .compact();
}
}
```



Cómo usarlo en tu Chat Controller

```
@RestController
```

```
@RequestMapping("/chat")
```

```
public class ChatController {
```

```
    private final JwtDelegationService jwtDelegationService;
```

```
    private final AcademiesApiClient academiesApiClient;
```

```
public ChatController(JwtDelegationService jwtDelegationService,
AcademiesApiClient academiesApiClient) {

    this.jwtDelegationService = jwtDelegationService;

    this.academiesApiClient = academiesApiClient;

}

@PostMapping("/ask")

public ResponseEntity<?>
processChatMessage(@RequestHeader("Authorization") String userToken,

                                                             @RequestBody ChatRequest
request) {

    // ❶ Extraer datos básicos del usuario del token recibido (p. ej.
email o id)

    String userEmail = extractEmailFromUserToken(userToken);

    String userRole = extractRoleFromUserToken(userToken);

    // ❷ Generar el token delegado corto

    String delegatedToken =
jwtDelegationService.generateDelegatedToken(userEmail, userRole);

    // ❸ Llamar al API de academias usando este token

    var response = academiesApiClient.getCourses(delegatedToken);
```

```
        return ResponseEntity.ok(response);
    }

    // Métodos auxiliares (puedes usar jwt.decode sin verificar para
    // extraer info)

    private String extractEmailFromUserToken(String token) {

        try {

            var claims = Jwts.parser()

                .setSigningKey("clave-supersecreta-api".getBytes()) //
                // la del API original

                .build()

                .parseClaimsJws(token.replace("Bearer ", ""))

                .getBody();

            return claims.getSubject();

        } catch (Exception e) {

            return "unknown";

        }

    }

    private String extractRoleFromUserToken(String token) {

        try {

            var claims = Jwts.parser()

                .setSigningKey("clave-supersecreta-api".getBytes())
```

```
        .build()

        .parseClaimsJws(token.replace("Bearer ", ""))

        .getBody();

        return (String) claims.get("role");

    } catch (Exception e) {

        return "user";

    }

}

}
```

Resultado

Cuando el chat haga una llamada al API de academias, enviará este header:

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

y dentro del token tendrás:

```
{

  "sub": "angel@example.com",

  "role": "profesor",

  "actor": "chat-backend",

  "iss": "chat-backend",

  "exp": 1739342400
```

```
}
```

→ El API de academias (Python) aceptará este token perfectamente, porque validará `iss = chat-backend` con `SECRET_CHAT`.



En resumen

Componente	Clave usada	iss	Aceptado por API
API de academias (tokens normales)	SECRET_API	academies-api	✓
Chat-backend (delegación)	SECRET_CHAT	chat-backend	✓

✓ Así consigues interoperabilidad total entre tu chat en Java y tu API en Python, y dejas todo listo para que más adelante migres a un IdP (solo cambiarás la fuente de emisión).

¿Quieres que te pase también un ejemplo rápido del `AcademiesApiClient` (en Spring) que añada automáticamente el `Authorization: Bearer <delegated_token>` a cada request HTTP al API Python?

Tú dijiste:

ese ejemplo que me dices es un ejemplo de como el backend chat debe llamar a los recursos del api academias?

ChatGPT Plus

Exactamente ✓

el ejemplo que te mencioné (el del `AcademiesApiClient`) es justo eso: un cliente HTTP dentro de tu backend-chat (Spring Boot) que se encarga de llamar a los endpoints del API de academias.

El objetivo es que el chat, cuando necesita obtener datos del API (por ejemplo cursos, profesores, horarios, etc.),

no tenga que reescribir lógica HTTP en cada sitio, sino que use una única clase centralizada que:

- Añade el header **Authorization: Bearer <delegated_token>**
- Envía la petición al API de academias (Python)
- Devuelve la respuesta al chat-backend.

Ejemplo básico: **AcademiesApiClient.java**

```
package com.tuempresa.chat.client;
```

```
import org.springframework.beans.factory.annotation.Value;
```

```
import org.springframework.http.*;
```

```
import org.springframework.stereotype.Component;
```

```
import org.springframework.web.client.RestTemplate;
```

```
@Component
```

```
public class AcademiesApiClient {
```

```
    private final RestTemplate restTemplate;
```

```
    private final String baseUrl;
```

```
    public AcademiesApiClient(@Value("${academies.api.url}") String  
baseUrl) {
```

```
        this.restTemplate = new RestTemplate();

        this.baseUrl = baseUrl;
    }

    public String getCourses(String delegatedToken) {

        String url = baseUrl + "/courses";

        HttpHeaders headers = new HttpHeaders();

        headers.set("Authorization", "Bearer " + delegatedToken);

        headers.setContentType(MediaType.APPLICATION_JSON);

        HttpEntity<Void> request = new HttpEntity<>(headers);

        ResponseEntity<String> response = restTemplate.exchange(

            url,

            HttpMethod.GET,

            request,

            String.class

        );

        return response.getBody();
    }
}
```

```
// ♦ Puedes añadir más métodos: getProfessors(), getSchedules(), etc.  
}
```



Cómo se usa

En tu **ChatController** (o en el servicio que maneje la lógica del chat):

```
@PostMapping("/ask")
```

```
public ResponseEntity<?> processChatMessage(@RequestHeader("Authorization")  
String userToken,
```

```
                                             @RequestBody ChatRequest  
request) {
```

```
    // ❶ Sacas info del usuario (email/rol)
```

```
    String userEmail = extractEmailFromUserToken(userToken);
```

```
    String userRole = extractRoleFromUserToken(userToken);
```

```
    // ❷ Generas el token delegado corto
```

```
    String delegatedToken =  
jwtDelegationService.generateDelegatedToken(userEmail, userRole);
```

```
    // ❸ Llamas al API de academias con ese token
```

```
    String response = academiesApiClient.getCourses(delegatedToken);
```

```
// ④ Devuelves la respuesta al usuario

return ResponseEntity.ok(response);

}
```

Configuración (por ejemplo, en `application.yml`)

`academies:`

`api:`

`url: https://api-academias.midominio.com`



Qué consigues esto

- ✓ Centralizas todas las llamadas HTTP al API en una sola clase.
- ✓ Todas las peticiones se envían con el token delegado corto (JWT emitido por el chat).
- ✓ Tu API (en Python) las validará perfectamente gracias a la validación doble que ya implementaste.
- ✓ Cuando migres al IdP centralizado, este cliente no cambiará: solo el método que genera el token.

Perfecto 💪 — entonces tu flujo queda todavía más limpio.

Vamos a centrarnos en cómo hacer esas llamadas al API de academias directamente desde Android (Kotlin),
usando el `access_token` que la app ya obtiene en el login.

👉 No necesitas ninguna parte web ni del backend-chat para esto:
solo la app Android y tu API Python de academias.



Resumen de tu escenario

- 1 El usuario se loguea desde Android → la app llama al endpoint **POST /login** del API de academias (Python).
- 2 El API valida credenciales (MySQL AWS) y devuelve:

```
{  
  
  "access_token": "eyJhbGciOiJIUzI1NiIs...",  
  
  "refresh_token": "eyJhbGciOiJIUzI1NiIs..."  
}
```

- 3 La app guarda ambos tokens (por ejemplo en **DataStore**).
- 4 Para llamar a cualquier endpoint del API (por ejemplo **/courses**, **/teachers...**), la app envía el header:

```
Authorization: Bearer <access_token>
```

- 5 El API valida el token (como ya hace ahora) y responde.



Ejemplo de llamada desde Android (Kotlin con Retrofit)

- 1 Dependencias en **build.gradle**

```
implementation("com.squareup.retrofit2:retrofit:2.9.0")  
  
implementation("com.squareup.retrofit2:converter-gson:2.9.0")  
  
implementation("com.squareup.okhttp3:logging-interceptor:4.11.0")
```

2 Crea tu cliente Retrofit

```
object ApiClient {

    private const val BASE_URL = "https://api-academias.midominio.com/"

    private val logging = HttpLoggingInterceptor().apply {

        level = HttpLoggingInterceptor.Level.BODY

    }

    private val client = OkHttpClient.Builder()

        .addInterceptor(logging)

        .addInterceptor { chain ->

            val original = chain.request()

            val token = TokenManager.getValidAccessToken() // obtienes el token guardado

            val request = original.newBuilder()

                .header("Authorization", "Bearer $token")

                .build()

            chain.proceed(request)

        }

        .build()

}
```

```
val instance: AcademiesApi by lazy {  
  
    Retrofit.Builder()  
  
        .baseUrl(BASE_URL)  
  
        .client(client)  
  
        .addConverterFactory(GsonConverterFactory.create())  
  
        .build()  
  
        .create(AcademiesApi::class.java)  
  
    }  
}
```

3 Define tu interfaz de endpoints

```
interface AcademiesApi {  
  
    @GET("courses")  
  
    suspend fun getCourses(): Response<List<CourseDto>>  
  
    @GET("teachers")  
  
    suspend fun getTeachers(): Response<List<TeacherDto>>  
  
    @POST("login")  
  
    suspend fun login(@Body request: LoginRequest): Response<LoginResponse>
```

```
}
```

4 Modelo de login y tokens

```
data class LoginRequest(  
  
    val email: String,  
  
    val password: String  
  
)
```

```
data class LoginResponse(  
  
    val access_token: String,  
  
    val refresh_token: String  
  
)
```

5 TokenManager (gestión local de tokens)

```
object TokenManager {  
  
    private var accessToken: String? = null  
  
    private var refreshToken: String? = null  
  
  
    fun saveTokens(access: String, refresh: String) {  
  
        accessToken = access  
  
        refreshToken = refresh  
  
    }  
  
}
```



```
}

fun getValidAccessToken(): String? {

    // Aquí puedes comprobar expiración y refrescar si hace falta

    return accessToken

}

}
```

6 Ejemplo de uso en una **ViewModel**

```
class CoursesViewModel : ViewModel() {

    private val api = ApiClient.instance

    fun loadCourses() {

        viewModelScope.launch {

            val response = api.getCourses()

            if (response.isSuccessful) {

                val courses = response.body()

                println("Cursos: $courses")

            } else {

                println("Error: ${response.code()}")

            }

        }

    }

}
```

```
        }

    }

}

fun login(email: String, password: String) {

    viewModelScope.launch {

        val response = api.login(LoginRequest(email, password))

        if (response.isSuccessful) {

            val tokens = response.body()

            TokenManager.saveTokens(tokens!!.access_token,
tokens.refresh_token)

        }

    }

}

}
```

Resultado

- Android gestiona sus tokens (**access** y **refresh**).
- Cada llamada al API de academias añade automáticamente el header **Authorization: Bearer <access_token>**.
- Tu API valida el token (como ya hace).

- No necesitas hacer ninguna llamada desde el chat-backend en este caso (Android → API directo).



Y más adelante...

Cuando quieras añadir el chat con delegación (OBO):

- Android seguirá autenticándose igual.
- Solo el chat-backend usará el token delegado para hablar con el API.
- Android no cambia nada.

¿Quieres que te muestre también cómo hacer que el **TokenManager** refresque automáticamente el **access_token** cuando le quede poco tiempo de vida (antes de llamar al API)? Así lo dejas ya robusto.

Tú dijiste:

si, eso lo haríamos solo en la capa de presentacion de android, correcto?

ChatGPT Plus

Exactamente  — lo has entendido perfecto.

👉 El refresco automático del token se hace solo en la capa de presentación Android, porque solo la app Android tiene el **refresh_token**.

Ni el chat-backend ni el API de academias deben guardar o usar el refresh token.



Resumen de responsabilidades

Componente	Qué hace con los tokens
------------	-------------------------

Android App

- ♦ Recibe **access_token** + **refresh_token** en el login.
- ♦ Guarda ambos de forma segura.
- ♦ Antes de cada llamada al API o al chat, revisa si el **access_token** está por expirar y, si es así, usa el **refresh_token** para pedir uno nuevo.

Chat-backend

- ♦ Recibe el **access_token** del usuario (válido).
- ♦ No guarda refresh tokens.
- ♦ Genera un token delegado interno corto cuando necesita hablar con el API.

API de academias

- ♦ Emite los tokens (login).
- ♦ Valida los **access_token** (sean del usuario o del chat).
- ♦ No guarda ni usa **refresh_token**.

Por qué esto es importante

- El **refresh_token** es muy sensible (sirve para obtener nuevos access tokens sin volver a hacer login).
- Por seguridad, solo el cliente propietario de la sesión (Android) debe tenerlo.
- Si el **refresh_token** se filtra a otro servicio, podría usarse para suplantar sesiones.

Implementación en Android (refresco automático)

Vamos a completar el **TokenManager** para que haga el refresco anticipado.

TokenManager.kt

```
object TokenManager {

    private var accessToken: String? = null

    private var refreshToken: String? = null

    private var expiresAt: Long = 0L // epoch seconds

    fun saveTokens(access: String, refresh: String, expiresInSeconds: Long)
    {

        accessToken = access

        refreshToken = refresh

        expiresAt = System.currentTimeMillis() / 1000 + expiresInSeconds

    }

    suspend fun getValidAccessToken(api: AcademiesApi): String? {

        val now = System.currentTimeMillis() / 1000

        val margin = 180L // 3 minutos

        // Si falta poco para expirar, refresca

        if (now + margin >= expiresAt) {

            val response = api.refreshToken(

                mapOf("refresh_token" to refreshToken)

            )

            saveTokens(response.access_token, response.refresh_token, response.expires_in)
```

```

        )

        if (response.isSuccessful) {

            val newTokens = response.body()!!

            saveTokens(newTokens.access_token, newTokens.refresh_token,
newTokens.expires_in)

        } else {

            // Token inválido -> usuario debe volver a iniciar sesión

            accessToken = null

            refreshToken = null

            expiresAt = 0L

            return null

        }

    }

    return accessToken

}
}

```

En tu **ApiClient.kt** (Retrofit Interceptor)

```

private val client = OkHttpClient.Builder()

    .addInterceptor(logging)

    .addInterceptor { chain ->

```

```
        val original = chain.request()

        val api = ApiClient.instance // acceso temporal al api

        val token = runBlocking { TokenManager.getValidAccessToken(api) }
// refresca si hace falta

        val request = original.newBuilder()

            .header("Authorization", "Bearer $token")

            .build()

        chain.proceed(request)

    }

    .build()
```

Endpoint **/refresh** en el API (Python)

Tu API de academias ya debería tener este endpoint para emitir un nuevo access token:

```
@app.post("/refresh")

def refresh_token(data: dict):

    refresh_token = data.get("refresh_token")

    if not refresh_token:

        raise HTTPException(status_code=400, detail="Falta refresh_token")
```

```
# validar el refresh_token y emitir uno nuevo

new_access = generate_access_token(user)

new_refresh = generate_refresh_token(user)

return {

    "access_token": new_access,

    "refresh_token": new_refresh,

    "expires_in": 900 # por ejemplo, 15 minutos

}
```



Resultado

- ✓ El usuario solo inicia sesión una vez.
- ✓ Android renueva su **access_token** automáticamente (sin molestar al usuario).
- ✓ Tu API sigue segura: el **refresh_token** nunca sale del dispositivo.
- ✓ El chat-backend solo recibe tokens válidos (nunca caducados).

¿Quieres que te muestre también cómo guardar estos tokens cifrados en Android (**EncryptedDataStore**) para mayor seguridad?